Docker Architecture and Lifecycle

AIM: *To understand Docker architecture and the container lifecycle, install Docker, and execute Docker commands to manage images and interact with containers.*

Objectives:

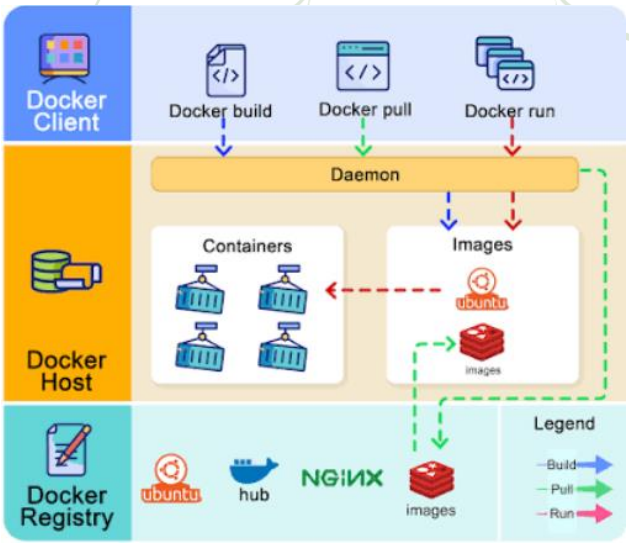
- *To understand the fundamentals of containerization using Docker.*
- *To study the architecture and components of Docker.*
- *To understand the lifecycle of a Docker container.*
- *To install and configure Docker on a system.*
- *To execute commonly used Docker commands for image and container management.*
- *To interact with running containers and observe their behavior.*

Description:

Docker is an open-source containerization platform that enables applications and their dependencies to be packaged into lightweight and portable containers. Unlike traditional virtual machines, containers share the host operating system kernel, making them more efficient in terms of resource utilization and startup time. Docker helps developers:

- *Build applications consistently across environments.*
- *Simplify deployment processes.*
- *Improve scalability and portability.*
- *Isolate applications from one another.*
- *Reduce dependency-related issues.*

Docker Architecture*:*



Docker follows a client-server architecture consisting of several components.

***Docker Client** :The Docker Client is the interface through which users execute Docker commands.*

- *docker run*
- *docker pull*
- *docker ps*

The client communicates with the Docker daemon to perform requested operations.

***Docker Daemon:** The Docker Daemon (dockerd) is responsible for:*

- *Building images.*
- *Managing containers.*
- *Handling Docker networks.*
- *Managing Docker volumes.*
- *Communicating with Docker clients.*

***Docker Host:** The Docker Host is the machine on which Docker is installed and running. It contains:*

• Docker Engine • Images • Containers Networks • Volumes

***Docker Registry** : A Docker Registry stores Docker images. Examples include:*

• Docker Hub • Private Docker Registry • Enterprise Container Registry

***Docker Images**:*

A Docker image is a read-only template used to create containers. Characteristics:

- *Contains application code and dependencies.*
- *Can be reused multiple times.*
- *Acts as a blueprint for containers.*
- *Stored locally or in registries.*

Examples: ubuntu, nginx, mysql, python

***Docker Containers**:*

A container is a running instance of a Docker image. Containers provide:

- *Process isolation.*

- *Application portability*
- *Lightweight execution.*
- *Rapid startup.*

Docker Container Lifecycle:

A Docker container passes through several states during its lifecycle.

Created: *The container has been created but is not running.*

Running: *The container is actively executing processes.*

Paused: *The container processes are temporarily suspended.*

Stopped: *The container is no longer running but still exists on the system.*

Removed: *The container has been deleted from the Docker host.*

Procedure :

1. *Download and install Docker Desktop or Docker Engine.*
2. *Verify that Docker has been installed successfully.*
3. *Study the major components of Docker architecture.*
4. *Understand the Docker container lifecycle.*
5. *Pull a Docker image from Docker Hub.*
6. *Create and start a container from the image.*
7. *View running and stopped containers.*
8. *Interact with a container using Docker commands.*
9. *Stop, restart, and remove containers.*
10. *Remove unused Docker images.*
11. *Record observations regarding image and container management.*

Execution:

Step 1: Install Docker

Download and install Docker Desktop from the official Docker website. After installation, start Docker Desktop and ensure the Docker service is running.

Step 2: Verify Docker Installation

Open Command Prompt, PowerShell, or Terminal and execute: docker --version A successful installation displays the installed Docker version.

Step 3: Verify Docker Service

Check whether Docker is functioning properly: docker info This command displays Docker Engine information and confirms communication with the Docker daemon.

Step 4: Download an Image

Pull the Ubuntu image from Docker Hub: docker pull ubuntu Docker downloads the required image layers and stores them locally.

Step 5: View Available Images

Display all downloaded images: docker images The command shows: • Repository name • Tag • Image ID • Creation date • Image size

Step 6: Create and Run a Container

Create and start a container using the Ubuntu image: docker run -it ubuntu bash Explanation: • -i : Interactive mode. • -t : Allocates a terminal. • ubuntu : Image name. • bash : Starts the Bash shell. The terminal prompt changes to indicate execution inside the container.

Step 7: Interact with the Container

Execute commands within the running container:

- *pwd*
- *ls*
- *whoami*

These commands demonstrate interaction with the containerized environment

Step 8: Exit the Container

Exit the running container: exit The container stops after exiting the shell session.

Step 9: View Container Lifecycle Status

Display all containers: docker ps -a

Observe the container status: • Up (Running) • Exited (Stopped)

This demonstrates lifecycle state transitions.

Step 10: Restart the Container

Start the stopped container again: docker start <container_id>

Verify the status: docker ps

Step 11: Access the Running Container

Connect to the container: docker exec -it bash This opens an interactive shell session within the container.

Step 12: Remove the Container

Stop the container if necessary: docker stop Remove it: docker rm This deletes the container from the Docker host.

Step 13: Remove the Image

Delete the image: docker rmi ubuntu This removes the image from local storage

Conclusion:

The experiment provided practical understanding of Docker architecture and containerization concepts. Docker images and containers were managed using standard Docker commands, and the complete container lifecycle—from creation to removal—was studied. The experiment demonstrated how Docker enables lightweight, portable, and efficient application deployment through container technology.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY స్వయం తేజస్విన్ భవ 1979

Docker file and Image Building

AIM: *To study Dockerfile instructions and use a Dockerfile to create a container image for a simple web application..*

Objectives:

- *To understand the purpose and structure of a Dockerfile.*
- *To study commonly used Dockerfile instructions.*
- *To create a simple web application that can be served using Nginx.*
- *To write a Dockerfile for the web application.*
- *To build a Docker image using the Dockerfile.*
- *To run a container from the newly created image.*
- *To access and verify the web application through a web browser*

Description:

A Dockerfile is a text file containing a sequence of instructions used by Docker to construct an image. Instead of manually configuring a container every time an application needs to be deployed, the required environment and application files can be described in a Dockerfile. A Dockerfile therefore acts as a reproducible set of instructions for creating a Docker image.

Dockerfile Instructions:

Instruction	Purpose
FROM	Specifies the base image for the new image
WORKDIR	Sets the working directory inside the image
COPY	Copies files from the build context into the image
ADD	Adds files or directories to the image
RUN	Executes a command while building the image
CMD	Specifies the default command executed when a container starts
ENTRYPOINT	Defines the primary executable for the container
EXPOSE	Documents the port intended to be used by the application
ENV	Defines environment variables
LABEL	Adds metadata to the image

Base Image:

The FROM instruction specifies the starting point for the image. For this experiment:

FROM nginx

The official Nginx image provides a ready-to-use web server environment.

***Copying Application Files**:*

The COPY instruction transfers files from the Docker build context into the image. For the sample application:

COPY index.html /usr/share/nginx/html

The HTML file is copied into the default directory from which Nginx serves static web content.

***Docker Image Building Process**:*

The Docker image is created using:

docker build -t myapp .

Here:

- *docker build starts the image-building process*
- *-t myapp assigns the name myapp to the image.*
- *. specifies the current directory as the build context.*

During the build, Docker reads the Dockerfile and executes its instructions to create the image.

***Running the Image as a Container**:*

After building the image, a container can be created using:

docker run -p 8080:80 myapp

The port mapping means: Host Port 8080 → Container Port 80

Therefore, requests sent to port 8080 on the host are forwarded to Nginx running on port 80 inside the container

Procedure :

1. Verify that Docker is installed and running.

2. Create a directory for the sample web application.

3. Create an HTML file containing the web-page content.

4. Create a file named Dockerfile in the application directory.

5. Add appropriate Dockerfile instructions.

6. Build a Docker image from the Dockerfile.

7. Verify that the newly created image is available locally.

8. Create and run a container using the image.

9. Map the container's web-server port to a port on the host system.

10. Open the application in a web browser and verify its output.

Execution:

Step 1: Create the Application Directory

Open PowerShell or Terminal and create a directory for the web application:

mkdir myapp

cd myapp

```
    Directory: C:\Users\Amol Pathak\Documents

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        07-10-2026     16:54                myapp
```

Step 2: Create the HTML File

Create a file named index.html.

For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web page is running inside a Docker container.</p>
</body>
</html>
```

Step 3: Create the Dockerfile

Inside the same directory, create a file named: Dockerfile

The Dockerfile should contain:

FROM nginx

COPY index.html /usr/share/nginx/html

The first instruction selects Nginx as the base image, while the second copies the application's HTML file into the Nginx web directory. The reference experiment uses this same minimal Dockerfile structure for the sample application.

```
    Directory: C:\Users\Amol Pathak\Documents\myapp


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         07-10-2026     16:59             49 Dockerfile
-a----         07-10-2026     16:57            197 index.html
```

Step 4: Verify Docker

Before building the image, verify that Docker is available: docker --version

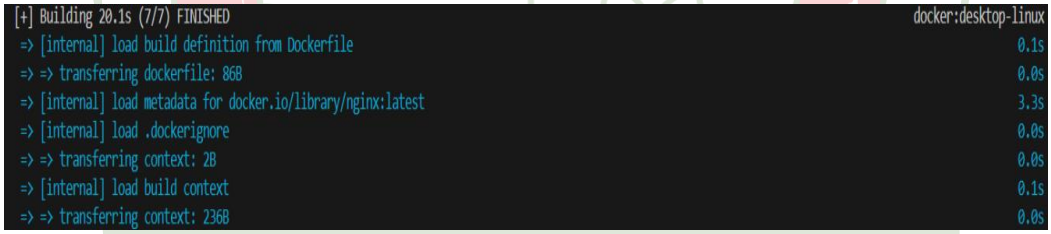
Optionally, obtain detailed Docker information using: docker info

Step 5: Build the Docker Image

From the directory containing the Dockerfile, execute:

docker build -t myapp .

Docker reads the Dockerfile and generates the myapp image. The build process should indicate successful completion. The reference execution shows the image being successfully built and tagged as myapp:latest. Figure 3: Successfully building the Docker image



Step 6: Verify the Image

List the locally available Docker images:

docker images

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
myapp:latest	d714a8d1e386	240MB	63.9MB	
postgis/postgis:15-3.3	a2fc46b52819	825MB	206MB	U

Step 7: Run the Container

Start a container from the newly created image:

docker run -p 8080:80 myapp

Here, port 8080 of the host is mapped to port 80 of the Nginx container. The reference experiment uses the same port mapping to run the myapp image.

```
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/07 11:39:12 [notice] 1#1: using the "epoll" event method
2026/10/07 11:39:12 [notice] 1#1: nginx/1.31.6
2026/10/07 11:39:12 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/10/07 11:39:12 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/10/07 11:39:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/07 11:39:12 [notice] 1#1: start worker processes
2026/10/07 11:39:12 [notice] 1#1: start worker process 29
2026/10/07 11:39:12 [notice] 1#1: start worker process 30
2026/10/07 11:39:12 [notice] 1#1: start worker process 31
2026/10/07 11:39:12 [notice] 1#1: start worker process 32
2026/10/07 11:39:12 [notice] 1#1: start worker process 33
2026/10/07 11:39:12 [notice] 1#1: start worker process 34
```

Step 8: Access the Web Application

Open a web browser and enter: http://localhost:8080

The HTML page created in Step 2 should be displayed. This confirms that:

1. The Dockerfile was processed successfully.

2. The Docker image was created.

3. A container was started from the image.

4. Nginx is serving the HTML file.

5. The application is accessible through the mapped host port.

Step 9: Verify the Running Container

Open another terminal and execute: docker ps

This displays the currently running containers. The output should show the myapp container and its port mapping

```
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                      NAMES
503f98f95df4   myapp     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    agitated_proskuria
ova
```

Conclusion:

The experiment provided practical understanding of Dockerfile-based image creation. The FROM and COPY instructions were used to construct an Nginx-based image containing a sample HTML application. The resulting image was built and executed as a container, demonstrating how Dockerfiles provide a repeatable method for packaging and deploying web applications.

Kubernetes Cluster

AIM: *Deploy a containerized application on Kubernetes cluster*

Description:

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications. Docker is used to package an application and its dependencies into a container image. In this experiment, a simple HTML web application is created and packaged into a Docker image using Nginx as the web server. The image is loaded into a local Kubernetes cluster managed by Minikube. A Kubernetes Deployment is created with two replicas to run the application in two Pods. Finally, a NodePort Service is configured to expose the application and make it accessible through a web browser

Execution:

Step 1: Start the Kubernetes cluster

Start Minikube and verify that the cluster is running. minikube start --driver=docker kubectl get nodes

Step 2: Create the application

Create a project directory and an index.html file containing a simple webpage with the message "Hello from Kubernetes!".

Step 3: Create the Dockerfile

Create a file named Dockerfile with the following instructions:

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Step 4: Build the Docker image

Build the application image using the following command:

docker build -t k8s-demo:v1 .

Verify that the image was created successfully:

docker images k8s-demo

Step 5: Load the image into Minikube

Make the Docker image available to the Kubernetes cluster.

minikube image load k8s-demo:v1

Step 6: Create a Kubernetes Deployment

Create a Deployment with two replicas of the application.

kubectl create deployment k8s-demo --image=k8s-demo:v1 --replicas=2

Verify the Deployment: kubectl get deployments

Step 7: Verify the Pods

Check whether both application Pods are running.

kubectl get pods

Both Pods should show Running status and 1/1 readiness.

Step 8: Expose the application

Create a NodePort Service to expose the application on port 80.

kubectl expose deployment k8s-demo --type=NodePort --port=80

Verify the Service:

kubectl get services

Step 9: Access the application

Run the following command to access the application through a browser:

minikube service k8s-demo

The webpage displays the message "Hello from Kubernetes!", confirming that the application is accessible.

Conclusion:

Thus, the containerized web application was successfully deployed on a Kubernetes cluster using Docker and Minikube. A Deployment was created with two replicas, the Pods were verified to be running, and a NodePort Service was used to expose the application. The application was accessed successfully through a web browser, demonstrating the basic process of container deployment and orchestration using Kubernetes.

Puppet

AIM: *To install and configure Pull based Software Configuration Management and provisioning tools using Puppet.*

Description:

Puppet is an open-source configuration management and provisioning tool used to automate the installation, configuration, and maintenance of systems. It follows a declarative approach in which the desired state of system resources is defined using Puppet manifests. In the pull-based model, the Puppet Agent periodically contacts the Puppet Server to retrieve its configuration catalog. The agent applies the catalog locally to ensure that the system matches the desired state. Puppet also supports idempotency, meaning repeated application of the same configuration does not make unnecessary changes when the system is already in the desired state. In this experiment, Puppet Agent and Puppet Server are installed in Ubuntu running on WSL. A simple Puppet manifest is created to ensure that a directory exists. The manifest is applied and tested for idempotency. Finally, the Puppet Agent contacts the Puppet Server, retrieves a catalog, and applies the configuration.

Execution:

Step 1: Update the package repositories

Update the Ubuntu package lists before installing Puppet.

```
sudo apt update
```

Step 2: Install Puppet Agent

Install the Puppet Agent package.

```
sudo apt install puppet-agent
```

Verify the installation:

```
puppet --version
```

Step 3: Install Puppet Server

Install the Puppet Server package.

```
sudo apt install puppetserver
```

Step 4: Start and verify the Puppet Server

Start the Puppet Server service.

sudo systemctl start puppetserver

Check its status:

sudo systemctl status puppetserver

The service should display active (running).

Step 5: Configure the Puppet Server hostname

Check the server hostname configured for the agent.

sudo puppet config print server --section agent

The configured hostname is puppet. Add the hostname mapping to /etc/hosts so the agent can resolve it locally.

echo "127.0.0.1 puppet" | sudo tee -a /etc/hosts

Verify the hostname resolution:

getent hosts puppet

Step 6: Verify the Puppet Server port

Check whether the Puppet Server is listening on port 8140

sudo ss -ltnp | grep 8140

The output should show a listening service on port 8140.

Step 7: Create a Puppet manifest

Create a manifest named first.pp.

nano first.pp

Enter the following Puppet code:

file { '/tmp/puppet-demo': ensure => directory, }

Save the file and exit the editor.

Step 8: Apply the Puppet manifest

Execute the manifest using the Puppet command.

sudo puppet apply first.pp

Puppet creates the directory /tmp/puppet-demo if it does not already exist.

Step 9: Verify the resource

Check whether the directory has been created.

ls -ld /tmp/puppet-demo

The output should display the directory and its permissions.

Step 10: Verify idempotency

Apply the same manifest again.

sudo puppet apply first.pp

Since the directory already exists in the desired state, Puppet should report that no resource changes are required.

Step 11: Execute the Puppet Agent

Run the Puppet Agent in test mode to initiate communication with the Puppet Server.

sudo puppet agent --test

The agent requests a catalog from puppet:8140, retrieves the catalog, and applies the configuration successfully.

Conclusion:

Thus, Puppet was successfully installed and configured as a pull-based Software Configuration Management and provisioning tool. A Puppet manifest was created and applied successfully, idempotency was verified, and communication between the Puppet Agent and Puppet Server was demonstrated.

Software Configuration Management

AIM: *To learn Software Configuration Management and provisioning using Puppet Blocks(Manifest, Modules, Classes, Function)*

Description:

Puppet provides a declarative language for defining the desired state of system resources. Its main building blocks help organize and reuse configuration code.

- ***Manifest****: A file containing Puppet code, usually with the .pp extension, that describes the desired configuration of resources.*
- ***Module:*** *A reusable collection of manifests and other supporting files organized in a standard directory structure.*
- ***Class:*** *A named block of Puppet code that groups related resources and can be included wherever required.*
- ***Function:*** *A reusable operation that accepts inputs and returns a value or performs a supported operation. Puppet provides built-in functions such as upcase() for transforming strings.*

In this experiment, a Puppet module is created with a class that manages a file. A manifest includes the class, and the built-in upcase() function is used to generate the file content. The configuration is then applied and verified.

Execution:

Step 1: Create the project directory

Create a directory for the Puppet experiment and move into it.

```
mkdir -p ~/puppet-blocks-lab/modules/webdemo/manifests
cd ~/puppet-blocks-lab
```

Step 2: Create a Puppet module

A module organizes related configuration code. The webdemo module contains a manifests directory and an init.pp file.

The directory structure is:

```
puppet-blocks-lab/
├── site.pp
└── modules/
    └── webdemo/
        └── manifests/
            └── init.pp
```

Step 3: Create a class inside the module

Create the module's main manifest.

```
nano modules/webdemo/manifests/init.pp
```

Add the following code:

```
class webdemo {
    file { '/tmp/puppet-blocks-demo.txt':
        ensure => file,
        content => upcase('puppet blocks demonstration'), } }
```

Save the file and exit the editor. Here, class webdemo defines a class, while the file resource ensures that the specified file exists with the required content.

Step 4: Understand the function

The built-in Puppet function upcase() converts a string into uppercase letters. For example:

```
upcase('puppet blocks demonstration')
```

The result is:

```
PUPPET BLOCKS DEMONSTRATION
```

This result is assigned to the file's content attribute.

Step 5: Create the main manifest

Create the main manifest that includes the module's class.

```
nano site.pp
```

Add the following code:

```
include webdemo
```

Save the file and exit the editor. The include statement declares the class so Puppet can apply the resources defined inside it.

Step 6: Apply the Puppet configuration

Run Puppet with the module path specified.

sudo puppet apply --modulepath=modules site.pp

Puppet locates the webdemo module, evaluates its class, executes the function, and creates the configured file.

Step 7: Verify the output

Display the generated file content.

cat /tmp/puppet-blocks-demo.txt

Expected output:

PUPPET BLOCKS DEMONSTRATION

Step 8: Verify idempotency

Apply the same configuration again.

sudo puppet apply --modulepath=modules site.pp

Puppet should report that no resource changes are required because the file already matches the desired state

Conclusion:

Thus, Software Configuration Management and provisioning using Puppet blocks were studied and implemented successfully. A manifest was used to include a class, a module was created to organize reusable configuration code, and the built-in upcase() function was used to generate the required file content. The experiment demonstrated the modular, reusable, and idempotent nature of Puppet configuration management.